Breaking and Entering

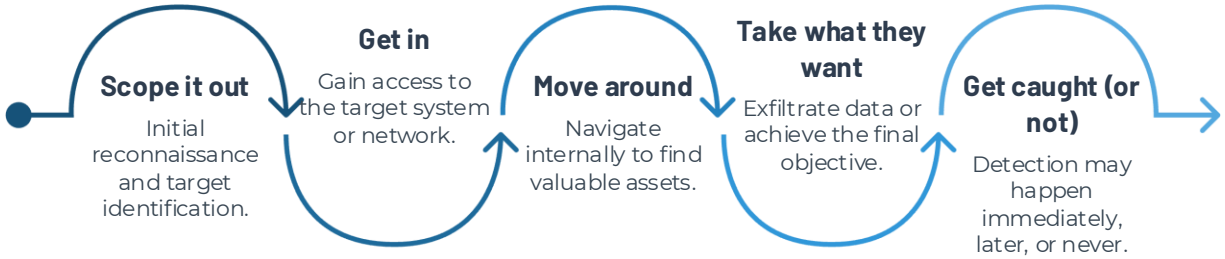
Digitally Speaking

A look at real ways attackers get in, and what we can do about it.



The Cyber Kill Chain

Break-ins follow a pattern, not a single moment:



This is the Cyber Kill Chain — how real attacks actually unfold, step by step.

STAGE 1 — RECONNAISSANCE

Scoping the Place Out



A burglar doesn't kick in the first door they see. They walk the neighborhood first — checking cameras, dogs, cracked windows.

Before trying anything, an attacker just **observes** — what software is running, how the app behaves, where the edges are.

Public tools, job postings, API responses, and normal app behavior all build a picture of your system before a single attack is launched.

Anything your app reveals about itself for free is information an attacker can use.



STAGE 1 — INFORMATION DISCLOSURE

Your App Talks More Than You Think

YSometimes the application tells attackers exactly what they need to know.

- Error messages
- API responses
- HTTP headers
- Public endpoints

Attackers piece together these clues to map your infrastructure, services, and potential weaknesses.

Every piece of public information helps an attacker build their attack strategy.

STAGE 1 — INFORMATION DISCLOSURE

The Attacker's Toolkit: Simple Reconnaissance

Attackers don't just guess; they gather clues from publicly available information.

Server Software Versions

When you visit a website, the server announces what software it's running (like "Apache 2.4"). Attackers can read this and look up known vulnerabilities in that exact version.

Clues from Job Postings and Social Media

A job posting says "We're hiring a Python/Django developer." Now the attacker knows the company uses Django and can research known vulnerabilities in that framework. LinkedIn, GitHub, and company posts reveal team names, cloud services, and tools.

What the App Sends and Gets

You open your browser's developer tools with `F12` and watch every request the app makes. An attacker can do the same thing, see the data being sent, and learn which endpoints and fields matter.

Trying the Obvious Doors

Recon doesn't stop at watching — it includes actively poking around, quietly.

01

Locate the target.

A DNS lookup is a simple, public query. Ask "what servers does rahvana.com use?" and the answer often reveals hidden infrastructure that was never meant to be found: staging.rahvana.com, admin.rahvana.com, dev.rahvana.com. These aren't linked anywhere. They're just... there in the DNS records, waiting to be discovered.





DIRECTORY DISCOVERY

Finding Hidden Entrances

Finding a server is only the beginning.Now the attacker checks what's behind each door.

Automated tools try hundreds of common page names developers frequently reuse: `/admin` · `/login` · `/dashboard` · `/.env` · `/backup`

Most return **404 Not Found**.

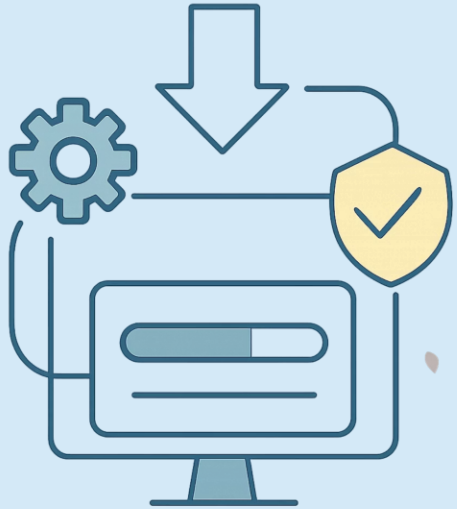
But it only takes one response to reveal an admin panel, forgotten backup, or internal dashboard. Nothing has been broken into yet. The attacker has simply found a door that was never meant to be obvious.

The lesson: if a page exists but isn't linked anywhere, a script can still find it by trying the obvious names.

STAGE 2 — ALTERNATIVE ENTRY

Supply Chain Attack: Trusting the Wrong Thing

One more way in, and it skips your users entirely — it targets something you already trust. Instead of attacking a company directly, an attacker compromises a tool or library that company already uses, and lets the company install the malicious update themselves.





STAGE 2

Getting In

Deliver

Now that an attacker knows what they're looking for, they need to actually get in. This means:

- Tricking a user into giving up their password
- Exploiting a weakness in the app to bypass login
- Finding a way to make the app do something it shouldn't

The next few slides show real ways this happens.

STAGE 2 — DELIVER | PHISHING & SOCIAL ENGINEERING

Stealing Someone's Keys

Sometimes the easiest vulnerability isn't software.

It's people.

Attackers create fake emails, fake login pages and fake IT requests designed to convince someone to hand over access.

Phishing Email

Fake login page captures your credentials.

Pretexting

Fake IT support calls asking for your password.

Baiting

USB drive with malware left in public.

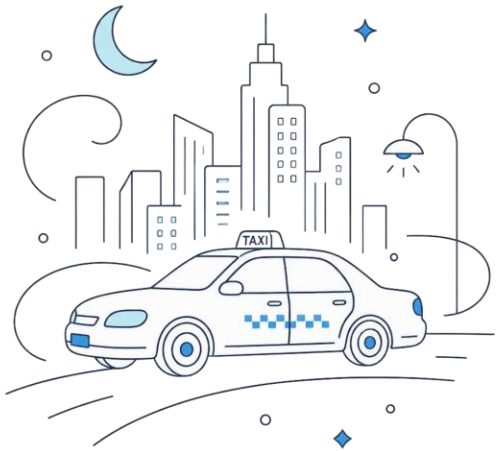
Watering Hole

Attacker compromises a trusted site your audience already uses and waits for them to visit.

⚠ **The weakness isn't technology it's trust**

Misleading the Visitor

Open Redirect



Users trust URLs that start with **rahvana.com**. An attacker exploits that trust.

The app has a redirect feature: `?redirect=dashboard` sends you to the dashboard after login. But if the app doesn't validate the redirect target, an attacker can change it to point anywhere.

They send: **rahvana.com/login?redirect=attacker-site.com**

You see **rahvana.com** in the URL and trust it. You log in. Then you're sent to a fake page that looks like Rahvana but isn't. By then, the attacker already has your credentials from the real login.

The app's own redirect feature becomes the weapon.

A taxi takes you to the address you gave it. If the meter is tampered with, the ride still looks real — but you end up somewhere else

BRUTE FORCE

Trying Every Key

An attacker targets the login page. They run a script that automatically tries thousands of common passwords: `password123`, `admin123`, `company2024`.

Each wrong guess returns “incorrect password.” But if there’s no rate limiting, there’s no limit to how many attempts they can make.

Rate limiting is the defense that blocks or slows login attempts after too many failures. Without it, the script keeps trying forever.

Eventually, one password matches and the attacker gets in.

The weakness: no defense against relentless guessing.

A login with no rate limiting is just a door nobody’s tried enough keys on yet.





SQL INJECTION

Fooling the Reception Desk

Tricking the Database

Some login forms are dumb about how they check your password — they just paste whatever you type directly into a command for the database. So instead of typing a password, you type something like `' OR 1=1 --`, and that little bit of punctuation actually changes what the database is told to do. It ends up just saying "yeah, that's valid" — because you didn't guess the password, you rewrote the question.

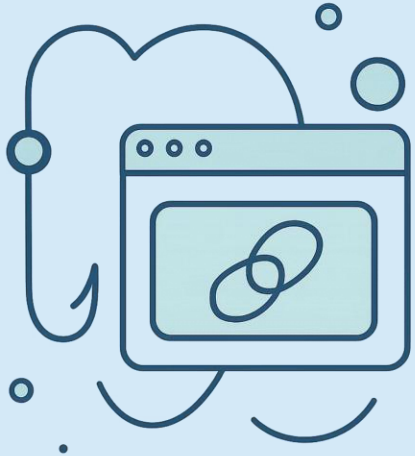
STAGE 2 — BREAK IN

Planting Something Behind

XSS (Cross-Site Scripting)

some websites take what you type — a comment, a review, a search term — and just display it back on the page, exactly as typed. If they don't check it first, you're not limited to typing normal text. You can type a tiny snippet of actual code instead. The website doesn't know the difference between "words" and "instructions" — so it just runs it, in the browser of anyone who later loads that page





Rewriting the Return Address

Host Header Injection

When you request a password reset, the app sends you an email with a reset link. That link should point to `rahvana.com`.

But the app builds that link from the request itself — specifically, the `Host` header, which tells the server what website the request came from.

An attacker sends a request with `Host: attacker-site.com`. The app uses that untrusted input to generate a password reset email with a link pointing to `attacker-site.com` instead.

The email looks official. You click the link. You reset your password on what looks like Rahvana. The attacker captures your new password.

The weakness: the app trusts user-controlled input to build links.



STAGE 3

MOVE AROUND

This is the stage where attackers navigate the system to find and access data they shouldn't be able to reach.

PATH TRAVERSAL

Can I Read Files I Shouldn't?



What is it?

Every file on a server has an address, known as a file path. If an application builds this path using user input without proper restrictions, an attacker can alter the path to reach files that were never intended to be accessible.

Imagine a filing cabinet with separate drawers for each person. You should only open your own drawer. But if the system doesn't check which drawer you're asking for, you can use `..` to climb out of your drawer and into someone else's.

Normal request:

`my-file.pdf` → opens your file

Malicious request

`../../other-user/file.pdf` → escapes your folder

A FILE'S LABEL ISN'T PROOF OF WHAT IT IS

Can I Upload Something Dangerous?



Checking a file's extension or declared type only checks something the uploader controls — so it is easy to fake. Real validation reads the actual content.

Someone renames a file to `passport.pdf`. If the system trusts the label alone, it gets accepted without looking inside.

The fix: Check real file content, not just the name. The code for this already exists — it just isn't switched on.

THE SERVER BECOMES AN UNWILLING INSIDE

Can I Reach Internal Systems?

SSRF



Say an app lets you set your profile picture from a URL — paste a link, the server fetches the image from wherever that link points.

Normally that link points somewhere out on the internet. But nothing stops it from pointing at the server's own internal systems, the ones never to be reached from the outside. The server doesn't know the difference. It just fetches whatever address it's given, and hand back what it finds.

So instead of an image, the attacker gets back internal data — configuration details, internal service responses, sometimes even credentials — stuff that was never supposed to be reachable by anyone outside the system.

The fix: block requests to internal addresses. Only allow a known, approved list.



A DIFFERENT GOAL

When the Goal Is Just to Break Things

DoS & DDoS:

Everything so far assumes the goal is getting in and taking something. Not always true — some attackers just want you offline.

DoS is one attacker flooding a site with requests until it can't respond. DDoS is the same idea from thousands of hijacked devices at once, so blocking one source doesn't help.

STAGE 4

ACHIEVING THE GOAL

Every previous step led here.

If attackers succeed they may steal:

- Personal documents
- Financial information
- Private customer records
- Internal business data

Real People

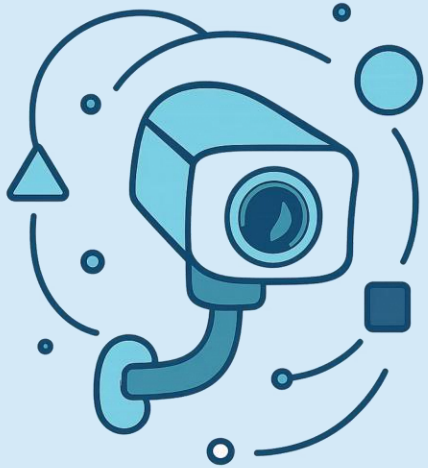
Real consequences.

STAGE 5

Would Anyone Notice?

Most systems don't watch closely enough for the small signs — repeated failed logins, one account pulling an unusual number of files, a sudden spike in errors. Detection doesn't stop a break-in. It's the difference between catching it in hours, or stumbling onto it by accident, months later.

KEY DISTINCTION: Prevention = close the door. Detection = notice if one got opened anyway.





You can't prevent every attack.

Focus on catching it fast.

The goal shifts from building impenetrable walls to quickly detecting and responding when the walls are inevitably breached.

WHAT DETECTION LOOKS LIKE:

- Alert on 5+ failed logins in 5 minutes (brute force attempt)
- Alert on one user accessing 100+ other users' files in an hour (unusual access pattern)
- Alert on sudden spike in 403 errors from one IP (someone probing for access)
- Alert on admin role changes outside normal business hours

THE FIX:

Set up basic alerting on obvious warning signs

So a problem gets noticed in hours, not discovered by accident months later

This is about visibility, not blocking — let the team know something suspicious is happening

THANK YOU

Catch it early,
and it stops being a problem.

Simple detection turns small warning signs into quick action.

